

Wrapper Functions Basic Writeup

Abstract

Wrapper functions are a common class of functions that provide an interface to one or more underlying functions or computational procedures with certain inputs preselected by the wrapper creator. Wrappers can greatly simplify the use of complex code, making it far more accessible to less experienced users. However, many possible modifications to the original code may become inaccessible, making edits not foreseen by the wrapper creator difficult to impossible for users. There are a few ways for wrapper creators to make future edits to their function easier. These approaches begin with making the function code more accessible for adjustment, but they can also provide a mechanism for users to view, edit, and save modified versions of the wrapper.

What are Wrapper functions

Basic description

Wrapper functions are a broad category of functions that typically aim to take a more complex task and create a simple way of recreating it, often by limiting the number of required inputs. To accomplish this, a more complex block of code is wrapped up in a function and many inputs have values selected by the creator. A few key inputs are left outward-facing to be adjusted by the user with the wrapper function. However, many inputs may be permanently unadjustable by the user. Common uses for wrapper functions include modeling functions, where a single function call may test numerous models, check assumptions, clean the data, and return what it considers the best-fitting model. Another common use is in complex graphics where there may be hundreds of lines of code, but only a couple are typically of interest to the user. Of particular interest is the large class of extensions that utilize wrappers for creating specific styles of ggplots.

Popular examples

A popular class of wrapper functions helps users create complex ggplots involving multiple layers with as little as one line of code (Wickham 2016). A major example of this style is `autoplot()`, which uses the class of the object given to it to choose an appropriate ggplot display. This allows users to quickly plot objects like time series, model objects, or dimension reduction results without needing to know all of the individual ggplot layers being created. The **survminer** package provides a more domain-specific example with `ggsurvplot()`, a wrapper that builds survival curves from fitted survival objects and can add common survival-analysis features such as confidence intervals, censoring marks, p-values, and risk tables (Kassambara et al. 2026). The **ggpubr** package takes a broader plotting-helper approach, providing functions such as `ggboxplot()` and `ggscatter()` that wrap common ggplot patterns behind simple arguments for labels, grouping, colors, and statistical annotations (Kassambara 2026). There are also packages created as companions for textbooks that often have wrapper functions to help create graphics like those seen in the book. The textbook **Time Series Analysis and Its Applications** (Shumway and Stoffer 2025) has its own package **astsa** with several wrappers for creating similar graphics, and for quickly determining models (Stoffer 2025). Even packages not focused on producing graphics may have one or two wrappers for creating specific visuals related to the goal of their package.

Another important class of wrapper functions is involved in modeling. The **fable** package provides time-series modeling functions that can take as little as a dataset, make numerous decisions for the user, and return what they determine to be the best model for the data (O'Hara-Wild et al. 2024).

Advantages

The clear advantage to creating a wrapper function is enabling users to create complex figures or well-fitted models with ease. For a user looking to complete a single application with as little friction as possible, this is ideal. With a wrapper function, the creator can enable beginner users to recreate their more advanced figures or models. This may be important when users have colleagues who need to replicate work in a similar style, since it reduces the expertise required and increases consistency. If none of the fixed inputs need to be changed, wrapper functions offer primarily upside. In some cases, what would have been hundreds of lines of code becomes one.

Outside of mere ease of use, wrapper functions can allow a user to lean on the experience of the creator of the function. If the creator is an expert in the area, their preselected input values or methods for determining them automatically may be more reasonable than anything an inexperienced user could guess. The user is less likely to violate unknown assumptions if the function checks those itself. Core code may have even been converted into C or other faster-running languages, making the function faster than anything the user could easily write themselves.

Issues

Issues can begin to crop up if an input that was fixed by the creator is something you need to adjust. It could be as simple as switching the line colors of a plot to be more colorblind-friendly. Depending on how the wrapper function was written, making minor adjustments can range from a slight inconvenience to a total nightmare as you try to navigate the depths of their package architecture searching for where the color of those lines was actually encoded. We call wrapper functions “self-contained” if the code controlling the intended customization is accessible and if inputs and output structure are well documented. Self-contained wrappers are easy to adjust because users can create a new wrapper with slightly modified code or add an input for a value that was previously fixed. Difficulties arise when the function is not self-contained. The wrapper may call many other functions, each of which could conceivably contain the code you need to adjust, requiring you to search through all of them. Outside of just calling other functions, the function could use S3, S4, or R6, giving it custom methods that could further adjust how the output looks. If how these more complex methods function is not clearly documented it can further obscure how the output is generated making it difficult to make adjustments. Also, if what you want to adjust is controlled by compiled C code, there may be nothing you can do outside of compiling new C code or finding a new package.

Beyond difficulty with modifying the function, wrapper functions can be very difficult to bug fix. The resulting error when something goes wrong can be very unclear, particularly if the error is in another function called by the wrapper function. The user often struggles to determine whether the issue is with their inputs or whether they are using the wrapper function incorrectly. The error may be fatally incomprehensible if the error is with a compiled C function.

Finally, if a user would like to use the output of the wrapper as part of some greater project, they may be unsure what they have actually done. (There’s plenty to be said about the user not really understanding their code and what they have really done, but is that something worth going down. My proposals in this paper don’t really do anything to fix that issue.)

Ways to combat Basic Issues

Package creators typically create wrapper functions to make their code as usable for users as possible. So, what steps exist to allow wrapper functions to be as simple and functional, but open to adjustment as possible?

Pre-existing methods

There are several pre-existing ways to adjust wrapper functions that create ggplots. One of them is the `gginnards` package which gives the user tools to move, append, and delete layers from ggplots to allow the user to make adjustments to already generated ggplots. This can be

valuable if a user needs to make certain adjustments like changing which layer is on top, or adding/removing a certain geom to the plot.

A tool used by many creators to make their wrapper functions more customizable is allowing a `...` argument. This can allow for many potential customizations to be passed into the wrapper function. Issues can arise however if the wrapper has several internal functions that share argument names so that `...` impacts several at the same time. Another older solution is the base function `edit()` which when given a function opens the code for the function in a pop-out window that allows for editing and then assigning the edited code. There are a few difficulties in using `edit()`, for instance if you try and save edited code but there is a mismatch in the number of parenthesis it will quit and delete all changes without saving anything. Also, if the adjusted code is saved to a function in the global environment, but it called packages that were internal to a package, it may no longer function.

One-off editing method

Our first proposed method is inspired by `edit`. One way to make a function adjustable is to add an argument that runs the wrapper normally by default but, when changed, prints its code into a new document. Adjustments to the code could be made live before running it in the original function environment, ensuring that any internal package functions still work. This would be helpful for very minor changes like switching the color of a line in a plot, or changing a couple of fixed numeric values. While this method is easy to implement and nice for single adjustments, it is inconvenient if a user would like to consistently make the same change when using the function a number of times.

Binding method

While the previous method is nice for minor one-off changes, it would be rather inconvenient if an adjustment was desired to be used regularly. This is especially true if you want to use the wrapper function many times, all with the same adjustment. To build on the previous method, you could instead bind the adjusted code to a new object in the working environment. The wrapper function could be adjusted to take in modified code and run it instead of its original. The user could then easily make their adjustment one time and continue to use their modified version of the original wrapper function. The wrapper function could even allow the name of the binding to be passed into the function itself and then assign it to the global environment after adjustments have been made. This creates a more semi-permanent solution, but the edited code living in the global environment requires the user to re-create it, or save it as an RDS, between sessions. It also requires the creator to add several arguments to every wrapper function they would like to be editable by the user.

Template method

The previous methods have both involved editing wrapper functions to have their typical functionality as well as arguments that give them editing functions. For a single wrapper this can be useful, but for a package containing multiple wrappers it may be cleaner to have a single editing function that works for all wrapper functions. In this method the code bodies of wrapper functions are stored in the package, and a dedicated templating function copies them into a new document. The user provides a name for the template, and when the edits are complete, the modified code is saved as an editable .R file rather than remaining only in the global environment. These templates allow for long-term use of modified code for the user, and even for templates to be shared between users if several would like to make the same adjustment. The only adjustment that needs to be made to the wrapper functions themselves in this method is allowing for a template to be provided and running the template code instead.

How to Implement

To give a running example of how this might work, we've created a wrapper function called `raincloud` that takes in a tibble with two variables, `value` and `class`. The default wrapper produces a violin plot with jittered observations, a narrow boxplot, and a median point.

```
set.seed(17)
a <- tibble::tibble(
  value = c(rnorm(2e2, 1), 4 * rbeta(2e2, 1, 4)),
  class = c(rep("a", 2e2), rep("b", 2e2))
)
```

```
raincloud_original(x = a)
```

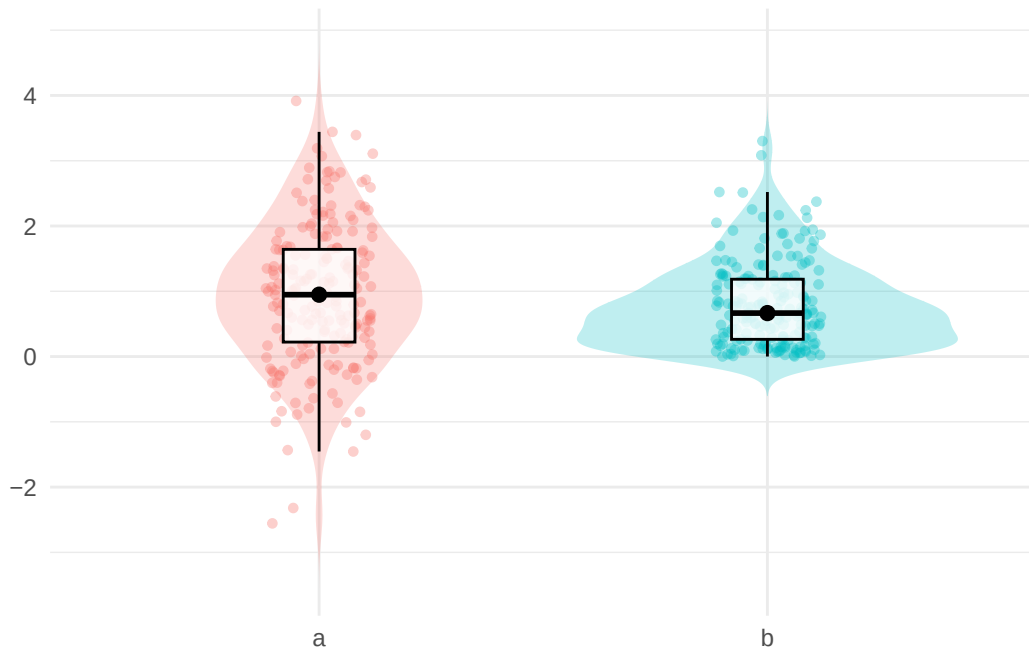


Figure 1: Default raincloud wrapper output.

The following methods rely on a copy of the function code stored in the package's `extdata` folder, where it can easily be read for editing.

One-off editing method

For the one-off editing method, the package creator first adds an argument such as `format` to the wrapper. This argument writes a copy of the template code into a new R script.

```
if (format == "plot") eval(code_output)
else if (format == "edit") {
  output <- glue(
    "# Explanation of how to edit the code here \n\n",
    "code_output$code <- ", "rlang::expr({{",
    system.file("extdata", "raincloud.R", package = "package name") |>
    readLines() |>
    paste(collapse = "\n"),
    "}})"
  )
  rstudioapi::documentNew(
    text = output,
```

```

    type = "r"
  )
  assign(
    "code_output",
    list(code = code_output, env = environment()),
    envir = .GlobalEnv
  )
}

```

This creates the editable script for the user and stores both the plot code and the wrapper's environment in `code_output`. The stored environment is important because it lets the edited code use objects created inside the wrapper function, such as processed versions of the input data.

The user first runs the plot in its default form, but finds that it's hard to see the points with the boxplot on top of them. To make an adjustment, they can rerun the function with `format` set to `"edit"`.

```

raincloud_original(x = a)
raincloud_original(x = a, format = "edit")

```

Running `raincloud_original(x = a, format = "edit")` opens an unnamed R script containing the following code. To make the points clearer, the user swaps the order of the jitter and boxplot layers.

```

code_output$code <- rlang::expr({print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
    geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
    geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
    geom_jitter(width = 0.12, alpha = 0.35, size = 1.2)+
    stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
    labs(x = NULL, y = NULL)+
    theme_minimal()+
    theme(legend.position = "none")
)})

```

After making the change, the user runs the edited block once to save the revised code object. Then they call `raincloud_adjust()` to evaluate the modified version in the original function environment.

```

raincloud_adjust()

```

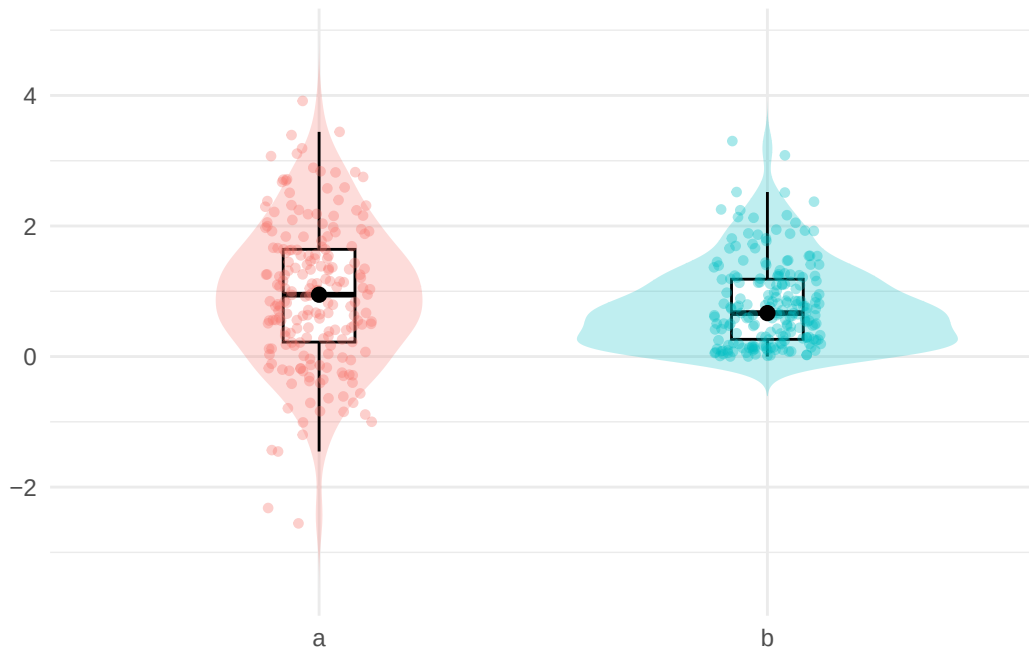


Figure 2: Original-method edit: the boxplot is drawn behind the jittered points.

Since the modified code lives in the default `code_output` object, creating a new version of the wrapper function will overwrite it. If the user wants several different edited versions with their own names, the next method is more convenient.

Binding method

The binding method keeps the same basic edit branch, but returns an object to be assigned as opposed to just saving to the global environment. As a slight improvement, a `bnd` class was created with a `print` method that takes the code of the object and evaluates it in the stored environment.

```
if (format == "plot") eval(code_output)
else if (format == "edit") {
  output <- glue(
    "# Fill in the object name to the left of $code \n\n",
    "$code <- ", "rlang::expr({{",
    system.file("extdata", "raincloud.R", package = "package name") |>
      readLines() |>
      paste(collapse = "\n"),
    "}})"
  )
}
```

```

)
rstudioapi::documentNew(
  text = output,
  type = "r"
)
bnd(list(code = code_output, env = environment()))
}

```

This means the user can save the returned object under a meaningful name, edit its `$code` slot, and then print the object to evaluate the modified code in the original function environment.

The user finds that points are a little clearer now that they are on top, but that the color blends into the violin plot background. They decide they would like to change the shape of the points and make them black so they stand out better. In case they want to make further adjustments to the code, they use the binding method and assign it to `black_points` in the global environment.

```

raincloud_binding(x = a)
black_points <- raincloud_binding(x = a, format = "edit")

```

The editable script again contains the plot code, but the wrapper returns an object with class `bnd`. The user fills in the object name on the left side of `$code`, edits the plot code, and runs the block.

```

black_points$code <- rlang::expr({print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
  geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
  geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
  geom_jitter(width = 0.12, alpha = 0.35, shape = 5, color = "black")+
  stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
  labs(x = NULL, y = NULL)+
  theme_minimal()+
  theme(legend.position = "none")
)})

```

Because `black_points` has a print method, the user can display the modified plot by evaluating the object itself.

```

black_points

```

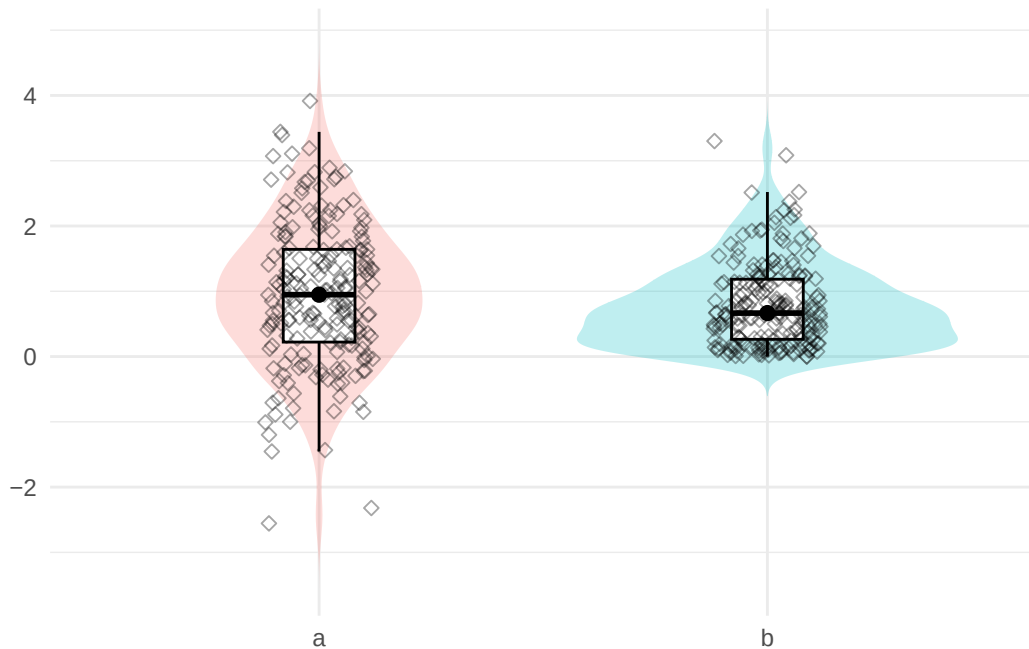


Figure 3: Binding-method edit: jittered points are drawn on top as black diamonds.

One limitation is that the modified code still lives inside the current R session unless the user saves it separately. For a more permanent solution, a user could use the following method.

Template method

For the template method, the creator uses an additional function that can generate new templates for every wrapper function in the package.

```
make_template <- function(fun = NULL, template_name = "basic") {
  if (fun %in% c("raincloud")) {
    src <- system.file("extdata", paste0(fun, ".R"), package = "package name")
    dest <- paste0(template_name, ".R")
    file.copy(src, dest, overwrite = FALSE)
    invisible(rstudioapi::navigateToFile(dest))
  }
}
```

The wrapper itself then needs only a small branch that runs the normal plot when no template is provided, or sources the user's edited template when one is provided.

```
raincloud_template <- function(x, template = NULL) {
  if (is.null(template)) {
    # default plotting code
  } else {
    source(template, local = TRUE)
  }
}
```

The template file contains the plot code that the user is meant to edit. It is more durable since modified code is stored in an editable .R file. Because the template is sourced locally, it can still use objects from inside the wrapper function.

The user decides that they would like to make one more adjustment, but decides to store it as a template for sharing. They name it `clearer_points` since all their adjustments make it easier to see the points than in the original wrapper function.

```
make_template("raincloud", "clearer_points")
```

This copies the package's raincloud plotting template into `clearer_points.R` and opens it for editing. The user can then apply the previous two changes and make one additional adjustment: increasing the jitter width from 0.12 to 0.35, giving the diamond points more horizontal room to spread out.

```
print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
    geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
    geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
    geom_jitter(width = 0.35, alpha = 0.35, shape = 5, color = "black")+
    stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
    labs(x = NULL, y = NULL)+
    theme_minimal()+
    theme(legend.position = "none")
)
```

Once the file is saved, the wrapper can use the modified template directly.

```
raincloud_template(a, "clearer_points.R")
```

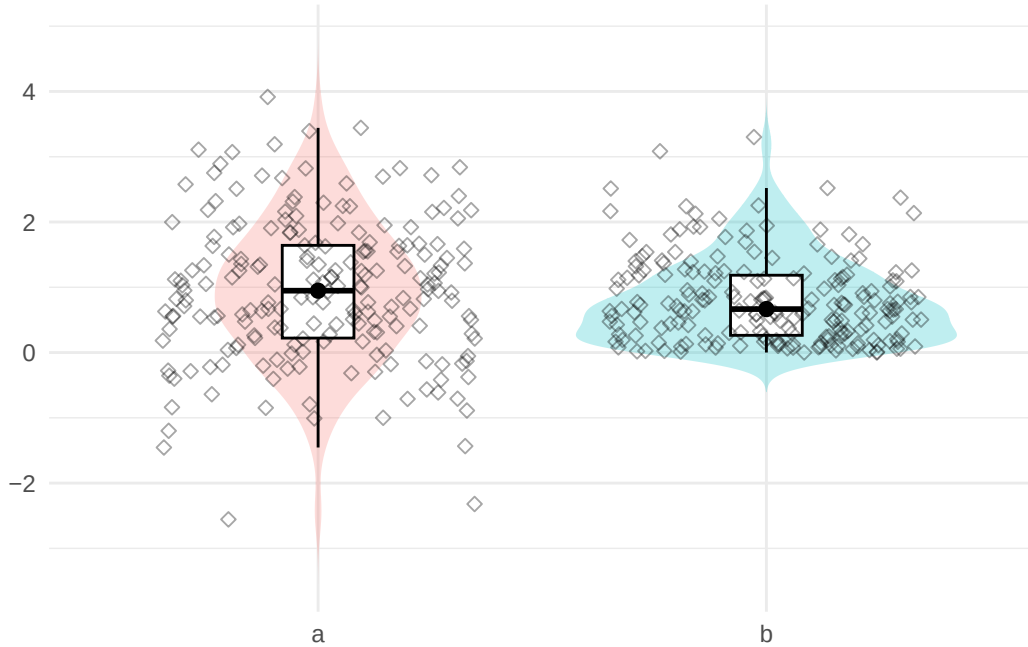


Figure 4: Template-method edit: previous edits plus wider jitter spacing.

A template for repeated measurements

A more substantial customization adapts the raincloud wrapper to repeated measurements from the same participants. Suppose each participant completes several trials in each of two conditions, and the desired display should show both the distribution of participant averages and each participant's change between conditions. The edited template, `paired_measurements.R`, expects an additional `participant` column identifying which observations belong to the same person; `class` identifies the condition, and `value` contains the trial response.

The template first averages trials within each participant and condition. It uses these averages for the violin, boxplot, and point layers, and adds lines connecting each participant's two averages. The distributions are grouped by condition, while the connecting lines are grouped by participant. One horizontal offset is generated for each participant and reused across both conditions and the point and line layers, ensuring that every line meets its corresponding points. These changes are stored together in the template so that the same procedure can be applied to another dataset with the required columns.

The example below uses simulated data for 24 participants, with six trials per participant in each condition. The factor levels in `class` determine the condition order, and setting a seed makes the horizontal offsets reproducible.

```
set.seed(17)
raincloud_template(paired_data, "paired_measurements.R")
```

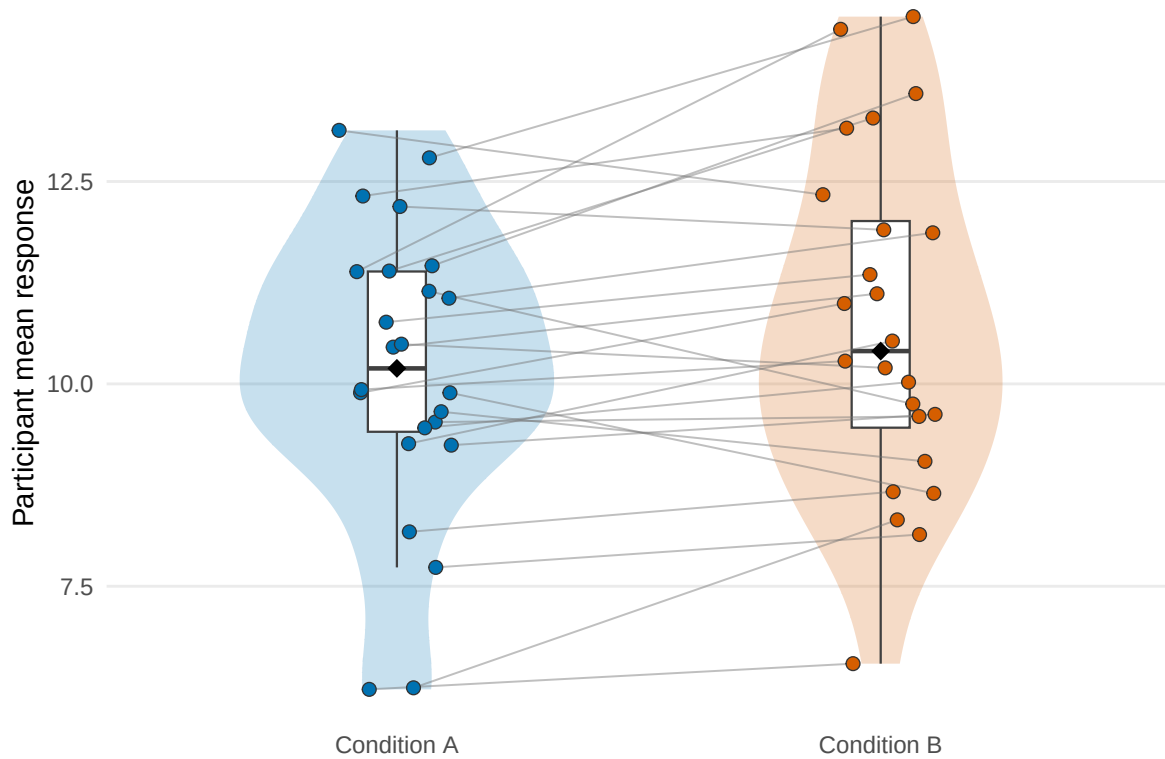


Figure 5: Paired-measurements template applied to simulated data. Each circle is a participant's mean over six trials; grey lines connect the same participant across conditions. Violins and boxplots summarize the 24 participant means in each condition, and black diamonds mark their medians.

Figure 5 is generated by executing the saved template through the existing wrapper. The template therefore retains the familiar wrapper call while coordinating changes to data preparation, grouping, and layer construction.

Conclusion

Considering the pros and cons of the methods described, we would have to recommend the template method as being the most valuable for users. The ability to not just adjust a function, but save those adjustments in an easily reusable and shareable way, is valuable in any collaborative setting where wrapper functions are being used.

For example, a research group might adapt a general-purpose raincloud wrapper to display repeated measurements from the same participants, as in Figure 5. The modified template first calculates participant-level summaries, then connects corresponding observations across conditions while retaining the group distributions. Producing this display requires coordinated changes to data preparation, grouping, and the positioning of multiple graphical layers. Saving these changes in a template allows the group to reuse and share a consistent plotting procedure across experiments without requiring the original package author to anticipate that particular combination of modifications.

The difficulty of implementation is not particularly higher than other simpler methods described. The largest barrier to entry for all methods described remains how self-contained a wrapper function is able to be, since code can only be edited if it is accessible to users.

References

- Kassambara, Alboukadel. 2026. *Ggpubr: 'Ggplot2' Based Publication Ready Plots*. <https://doi.org/10.32614/CRAN.package.ggpubr>.
- Kassambara, Alboukadel, Marcin Kosinski, and Przemyslaw Biecek. 2026. *Survminer: Drawing Survival Curves Using 'Ggplot2'*. <https://doi.org/10.32614/CRAN.package.survminer>.
- O'Hara-Wild, Mitchell, Rob Hyndman, and Earo Wang. 2024. *Fable: Forecasting Models for Tidy Time Series*. <https://doi.org/10.32614/CRAN.package.fable>.
- Shumway, Robert H., and David S. Stoffer. 2025. *Time Series Analysis and Its Applications: With r Examples*. 5th ed. Springer Texts in Statistics. Springer Cham. <https://doi.org/10.1007/978-3-031-70584-7>.
- Stoffer, David. 2025. *Asts: Applied Statistical Time Series Analysis*. <https://doi.org/10.32614/CRAN.package.astsa>.
- Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>.